Supplement: Evaluating Satisfiability Solving with LLMs

LEIZHEN ZHANG, University of Louisiana at Lafayette, USA
SHUHAN CHEN, East China Normal University, China
SHENG CHEN, University of Louisiana at Lafayette, USA

CCS Concepts: • **Theory of computation** → **Constraint and logic programming**; • **Computing methodologies** → *Artificial intelligence*; • **Software and its engineering** → *Software verification and validation.*

Additional Key Words and Phrases: large language models, satisfiability solving, SAT, 2-SAT, 3-SAT, logical reasoning, paired formulas, accurate differentiation rate, Vertex Cover, 3D packing

ACM Reference Format:
Leizhen Zhang, Shuhan Chen, and Sheng Chen. 2026. Supplement: Evaluating Satisfiability Solving with LLMs. *Proc. ACM Softw. Eng.* 3, FSE, Article FSE202 (July 2026), 19 pages. https://doi.org/10.1145/3808209

1 Mathematical Properties of ADR

In this section, we provide a rigorous derivation of the mathematical properties of the *Accurate Differentiation Rate (ADR)*. We show its relationship to conventional metrics such as accuracy, recall, and precision, establish tight upper and lower bounds, and argue why ADR serves as a stricter and more reliable measure of reasoning ability.

1.1 Setup and Notation

Consider M paired instances $\{(F_i^{\text{SAT}}, F_i^{\text{UNSAT}})\}_{i=1}^{M}$, each consisting of:

- one satisfiable formula F_i^{SAT},
- one unsatisfiable formula F_i^{UNSAT}.

For each pair, define binary correctness indicators:

$$A_i = \mathbf{1}\{\hat{y}(F_i^{\text{SAT}}) = \text{SAT}\}, \qquad B_i = \mathbf{1}\{\hat{y}(F_i^{\text{UNSAT}}) = \text{UNSAT}\}.$$

Then:

$$r_S = \frac{1}{M}\sum_{i=1}^{M} A_i, \quad r_U = \frac{1}{M}\sum_{i=1}^{M} B_i, \quad \text{ADR} = \frac{1}{M}\sum_{i=1}^{M} A_i B_i.$$

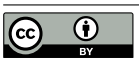
The per-instance accuracy over all $2M$ examples is:

$$\text{Acc} = \frac{1}{2M}\sum_{i=1}^{M}(A_i + B_i).$$

Lemma 1.1 (Accuracy identity).

$$\text{Acc} = \tfrac{1}{2}(r_S + r_U)$$

Authors' Contact Information: Leizhen Zhang, University of Louisiana at Lafayette, Lafayette, USA, leizhen.zhang1@louisiana.edu; Shuhan Chen, East China Normal University, Shanghai, China, 10225102449@stu.ecnu.edu.cn; Sheng Chen, University of Louisiana at Lafayette, Lafayette, USA, sheng.chen@louisiana.edu.



ACM 2994-970X/2026/7-ARTFSE202
https://doi.org/10.1145/3808209

1.2 Upper Bounds of ADR

Theorem 1.2 (ADR bounded by recalls). *For any paired dataset,*

$$\mathrm{ADR} \le \min(r_S, r_U)$$

Proof. For each pair, $A_iB_i \le A_i$ and $A_iB_i \le B_i$. Summing and normalizing proves the result.

Theorem 1.3 (ADR bounded by accuracy).

$$\mathrm{ADR} \le \mathrm{Acc}$$

Proof. For each pair, $A_iB_i \le \frac{1}{2}(A_i + B_i)$. Summing gives ADR $\le$ Acc.

1.3 Lower Bound of ADR

Lemma 1.4 (ADR lower bound).

$$\mathrm{ADR} \ge r_S + r_U - 1.$$

Equivalently,

$$\mathrm{ADR} \ge \max(0, r_S + r_U - 1)$$

Corollary 1.5 (ADR vs. accuracy). *Since* $\mathrm{Acc} = \frac{1}{2}(r_S + r_U)$,

$$\mathrm{ADR} \ge \max(0, 2\mathrm{Acc} - 1)$$

1.4 Independence Approximation

Theorem 1.6 (ADR under independence). *Let* $A = \{F^{SAT}$ *correct*$\}$ *and* $B = \{F^{UNSAT}$ *correct*$\}$. *Then*

$$\mathrm{ADR} = P(A \cap B) = r_S r_U + \mathrm{Cov}(\mathbf{1}_A, \mathbf{1}_B).$$

In particular, if A and B are independent,

$$\mathrm{ADR} = r_S r_U$$

Proof. By linearity of expectation,

$$\frac{1}{M}\sum A_iB_i = E[\mathbf{1}_A\mathbf{1}_B] = P(A)P(B) + \mathrm{Cov}(\mathbf{1}_A, \mathbf{1}_B).$$

If A and B are independent, the covariance vanishes.

1.5 Summary of Bounds

Theorem 1.7 (ADR inequality chain). *For any paired dataset,*

$$\max(0, r_S + r_U - 1) \;\le\; \mathrm{ADR} \;\le\; \min(r_S, r_U) \;\le\; \mathrm{Acc} = \tfrac{1}{2}(r_S + r_U)$$

This chain shows that ADR is a stricter measure than accuracy: it requires *both* sides of each pair to be correct, rather than only one. Consequently, ADR resists dataset imbalance and prediction bias, and more faithfully captures true reasoning competence.

1.6 ADR vs. Traditional Metrics: Essential Differences

- **Accuracy (per instance).** Strongly affected by class distribution [Manning 2008; Powers 2020]; even if one side is always right and the other always wrong, accuracy can still be 0.5, which looks "not too bad." **Fails to reflect pairwise discrimination ability.**
- **Precision/Recall/F1 (require positive class).** Once SAT is defined as the positive class, in SAT-dominated regions the metric can be inflated by "always predicting SAT." [Powers 2020] **Completely blind to the UNSAT side.**

- **ADR (per pair).** Independent of distribution, forces both sides to be correct simultaneously; **directly measures whether the model can distinguish the pair.**

In one sentence: **Accuracy/F1 mean "average good enough," while ADR means "both sides of the same pair must be correct."**

1.7 Toy Counterexamples (More Intuitive)

(1) Always Predict SAT.. $r_S = 1$, $r_U = 0$. Then Acc = 0.5, but ADR = 0. Accuracy looks decent, but ADR pinpoints the failure: cannot distinguish paired differences.

(2) Strong on One Side, Moderate on the Other. $r_S = 1.0$, $r_U = 0.6$. Then ADR ≤ 0.6, and the lower bound $1.0 + 0.6 - 1 = 0.6$, so ADR = 0.6. Meanwhile, Acc $= \frac{1.0+0.6}{2} = 0.8$. **Conclusion:** Accuracy = 0.8 looks "good," but ADR reveals the real bottleneck of pairwise correctness is only 0.6.

(3) Anti-Calibrated (Always Flip Within Pair). Every pair "gets exactly one side correct" $\Rightarrow$ Acc = 0.5, but ADR = 0. ADR correctly identifies total failure to capture decisive edits.

1.8 Practical Insights

- **Why more suitable for our task?** This paper emphasizes the ability to distinguish *minimal structural edits*. ADR directly checks whether the SAT/UNSAT pair is simultaneously classified correctly, naturally avoiding α-driven class imbalance bias.
- **Connection to witness validity.** In experiments, models with higher ADR also tend to produce more often *valid satisfying assignments or constructions [Biere et al. 2009]* (e.g., packing/cover solutions). This shows ADR is closer to "structural reasoning" rather than label-guessing.
- **When to prefer ADR?** Whenever the evaluation target is pairwise/counterfactual discrimination, or when dataset distribution drifts with conditions (e.g., α), ADR should be prioritized. Accuracy/F1 can be reported as supplementary but cannot stand alone as evidence of reasoning.

1.9 One-Sentence Summary

ADR is a **pairwise, symmetric, distribution-independent** metric that requires models to get both SAT and UNSAT of the same pair correct. It effectively suppresses bias and opportunism, highlighting true **structural discrimination and reasoning ability**. Mathematically, ADR is upper-bounded by $\min(r_S, r_U)$ and Acc, and lower-bounded by $\max(0, r_S + r_U - 1)$. These inequalities explain why ADR is stricter than Accuracy/F1 and better reveals the model's true reasoning capacity.

2 ADR vs. MCC: A Mathematical Comparison

In this section we rigorously compare the *Accurate Differentiation Rate (ADR)* with the *Matthews Correlation Coefficient (MCC)* in the paired-evaluation setting. We derive their closed-form relationship, express MCC in terms of pair-level decomposition, and highlight why ADR provides a clearer and more reliable measure of pairwise reasoning ability.

2.1 Pair Decomposition

Let M be the number of SAT/UNSAT pairs. For each $i = 1, \ldots, M$:

$$A_i = \mathbf{1}\{\hat{y}(F_i^{\mathrm{SAT}}) = \mathrm{SAT}\}, \quad B_i = \mathbf{1}\{\hat{y}(F_i^{\mathrm{UNSAT}}) = \mathrm{UNSAT}\}.$$

Define the pair-level counts:

$$\begin{aligned} n_{11} &= \#\{i : A_i = 1, B_i = 1\}, && \text{(both correct)} \\ n_{10} &= \#\{i : A_i = 1, B_i = 0\}, && \text{(SAT-only correct)} \\ n_{01} &= \#\{i : A_i = 0, B_i = 1\}, && \text{(UNSAT-only correct)} \\ n_{00} &= \#\{i : A_i = 0, B_i = 0\}, && \text{(both wrong).} \end{aligned}$$

Thus $M = n_{00} + n_{01} + n_{10} + n_{11}$, and

$$\mathrm{ADR} = \frac{n_{11}}{M}, \quad r_S = \frac{n_{10} + n_{11}}{M}, \quad r_U = \frac{n_{01} + n_{11}}{M}, \quad \mathrm{Acc} = \tfrac{1}{2}(r_S + r_U).$$

2.2 Closed-Form Expression of MCC

Theorem 2.1 (MCC in terms of recalls [Matthews 1975]). *On the balanced paired dataset, MCC can be expressed solely in terms of the recalls r_S and r_U:*

$$\mathrm{MCC} = \frac{r_S + r_U - 1}{\sqrt{(r_S + 1 - r_U)(r_U + 1 - r_S)}}.$$

Proof. Substituting

$$\mathrm{TP} = Mr_S, \quad \mathrm{TN} = Mr_U, \quad \mathrm{FP} = M(1 - r_U), \quad \mathrm{FN} = M(1 - r_S)$$

into the MCC definition and simplifying yields the stated result.

Corollary 2.2 (Undefined cases of MCC [Chicco and Jurman 2020]). *MCC is undefined if and only if $(r_S, r_U) \in \{(1, 0), (0, 1)\}$, i.e. the model always predicts one class. In contrast, ADR is always defined and equals 0 in these cases.*

2.3 Pair-Structured Expression

Lemma 2.3 (MCC in terms of ADR, β, and δ). *Define*

$$\delta = \tfrac{n_{10} - n_{01}}{M}, \qquad \beta = \tfrac{n_{00}}{M}.$$

Then

$$\mathrm{MCC} = \frac{\mathrm{ADR} - \beta}{\sqrt{1 - \delta^2}}. \tag{1}$$

Proof. Note that

$$r_S + r_U - 1 = \tfrac{n_{11} - n_{00}}{M} = \mathrm{ADR} - \beta,$$

and

$$r_S + 1 - r_U = 1 + \delta, \quad r_U + 1 - r_S = 1 - \delta.$$

Substituting into Theorem 2.1 proves the claim.

Corollary 2.4 (Sign condition). *From Lemma 2.3,* $\mathrm{MCC} > 0$ *if and only if* $\mathrm{ADR} > \beta$, *i.e. the proportion of both-correct pairs exceeds that of both-wrong pairs.*

2.4 Interpretation and Example

Equation (1) shows that MCC conflates joint correctness (ADR) with both-wrong frequency (β) and class asymmetry (δ). ADR, by contrast, isolates the joint-correct rate n_{11}/M, independent of δ.

Example. Let $r_S = r_U = 0.8$. Then Theorem 2.1 gives MCC = 0.6 regardless of joint distribution. But ADR satisfies

$$0.6 = \max(0, r_S + r_U - 1) \;\le\; \text{ADR} \;\le\; \min(r_S, r_U) = 0.8.$$

Thus two models can share the same MCC (0.6) yet have ADR = 0.6 (errors all one-sided) or ADR = 0.8 (errors all joint), revealing different discrimination abilities.

2.5 Summary

Theorem 2.5 (ADR vs. MCC). *On the balanced paired dataset,*

$$\text{MCC} = \frac{\text{ADR} - \beta}{\sqrt{1 - \delta^2}}, \quad \beta = \tfrac{n_{00}}{M}, \quad \delta = \tfrac{n_{10} - n_{01}}{M}.$$

Hence:

(1) ADR is always defined and directly measures joint correctness.
(2) MCC is undefined under class-collapse and mixes ADR with β and δ.
(3) Two models with identical MCC can differ in ADR, making ADR a stricter probe of pairwise reasoning.

3 VERTEX COVER, EQUIVALENT OR NOT EQUIVALENT?

3.1 From CNF to Graph: Motivation and Hypothesis

A central claim of this paper, reiterated in Section 1 (Introduction), is that *SAT solving is a foundational reasoning capability for LLMs*. Because a wide range of tasks in AI, program analysis, and combinatorial optimization reduce to SAT, an LLM's performance on SAT is an informative proxy for its potential to generalize across diverse reasoning problems. At the same time, SAT itself admits standard polynomial-time reductions to other canonical problems—notably *Vertex Cover*. If LLMs genuinely capture the *logical structure* underlying SAT, then their behavior should be *representation-invariant*: given an instance expressed as a CNF formula or as its graph-theoretic reduction, the model ought to reach consistent conclusions.

Representation-Invariance Question. Experiment 4 is designed to test precisely this question: ***RQ6:*** *Do LLMs reason about satisfiability in a manner that is stable across equivalent representations, or are apparent successes tied to the surface form of CNF?* Concretely, we take the paired SAT/UNSAT formulas from Experiment 3 in Section 5 (Refined Understanding with Paired Formulas) and apply a standard reduction from CNF-SAT to Vertex Cover, producing graph instances (V, E) with a bound k such that the formula is satisfiable iff the constructed graph admits a vertex cover of size $\le k$. We then prompt LLMs on the graph representation to decide the existence of such a cover and, when answering `YES`, to output a concrete vertex set.

Evaluation Plan. This setup allows us to assess two aspects of reasoning: (i) *task performance* on graphs (Accuracy/Precision/Recall/F1), and (ii) *cross-representation consistency* between CNF and graph inputs, i.e., whether categorical decisions (SAT vs. UNSAT vs. YES vs. NO) and, when applicable, the structural content of the prediction (returned cover sets) align across the two representations. High consistency would support the view that LLMs are leveraging common logical invariants rather than superficial input patterns; conversely, a sharp drop when moving from CNF to graphs would suggest representation- sensitive heuristics rather than genuine reasoning.

The remainder of this section presents the prompting protocol, datasets, and metrics, followed by results and analysis.

3.2 Reduction from CNF to Graph

It is well known that propositional formulas in conjunctive normal form (CNF) can be reduced to graph-based formulations of classical traditional NP-complete problems. In particular, CNF-SAT can be polynomially reduced to the *Vertex Cover* problem, which asks whether there exists a set of vertices of size at most k that covers all edges in a given undirected graph. Specifically, a CNF formula can be transformed into a directed acyclic graph (DAG) or equivalently into an undirected bipartite graph, where clauses and literals are represented as nodes and edges capture their relationships. If a CNF formula is satisfiable, then there exists a corresponding vertex cover of bounded size in the constructed graph. This reduction provides a graph-theoretic perspective on satisfiability and allows SAT solving to be studied through the lens of combinatorial optimization.

3.3 Experiment 5 setup.

Building on the paired SAT/UNSAT formulas from Experiment 3, we applied standard reductions to generate their corresponding graph instances for the vertex cover problem. Each graph was then provided to the LLMs with a structured prompt designed to query for a vertex cover of size at most k. An example of the prompt template is shown below:

```
def make_vc_prompt_core(V, E, k):
    V_str = "{ " + ", ".join(str(v) for v in V) + " }"
    E_str = "{ " + ", ".join("{" + f"{u}, {v}" + "}" for (u, v) in E) + " }"
    return (
      "You are solving a Vertex Cover decision task.\n"
      f"Graph: V={V_str}, E={E_str}\n"
      f"Question: Is there a vertex cover C with |C| <= {k}?\n\n"
      "OUTPUT (one-line JSON only):\n"
      '  {"answer":"YES"|"NO","cover":[...],"explain":"..."}\n'
      "Rules:\n"
      f"  - Return YES only if you can list C (|C|<= {k}) that covers every edge.\
    n"
      "  - Otherwise return NO and set cover=[].\n"
      "  - No extra text outside the JSON.\n"
    )
```

Listing 1. Core prompt for Vertex Cover (abbreviated). This listing shows only the decision-critical instructions and output schema. See Appendix 6 for the full prompt (edge-case rules, formatting guards) and the deterministic checker.

Each instance was given in the form (V, E) with vertex set V, edge set E, and the bound k. The model was required to output a JSON object specifying whether a vertex cover of size $\leq k$ exists, and, if so, to list the corresponding cover set.

3.4 Evaluation on Graph Inputs

Traditional metrics on graphs (Figure 1 (a–d)). Across Accuracy, Precision, Recall, and F1, most models operate near the random-guessing regime when asked to decide whether a small graph admits a vertex cover of size $\leq k$. Two notable exceptions are `GPT-5` and, to a lesser extent, `deepseek-reasoner`:

- **Most models:** Accurcay curves remain clustered around ≈ 0.5 and degrade with N, indicating little signal beyond chance.
- **`GPT-5`:** achieves near-perfect scores for small instances (N=5, 8, 10), but collapses as N grows, mirroring the scalability trend observed on CNF inputs.

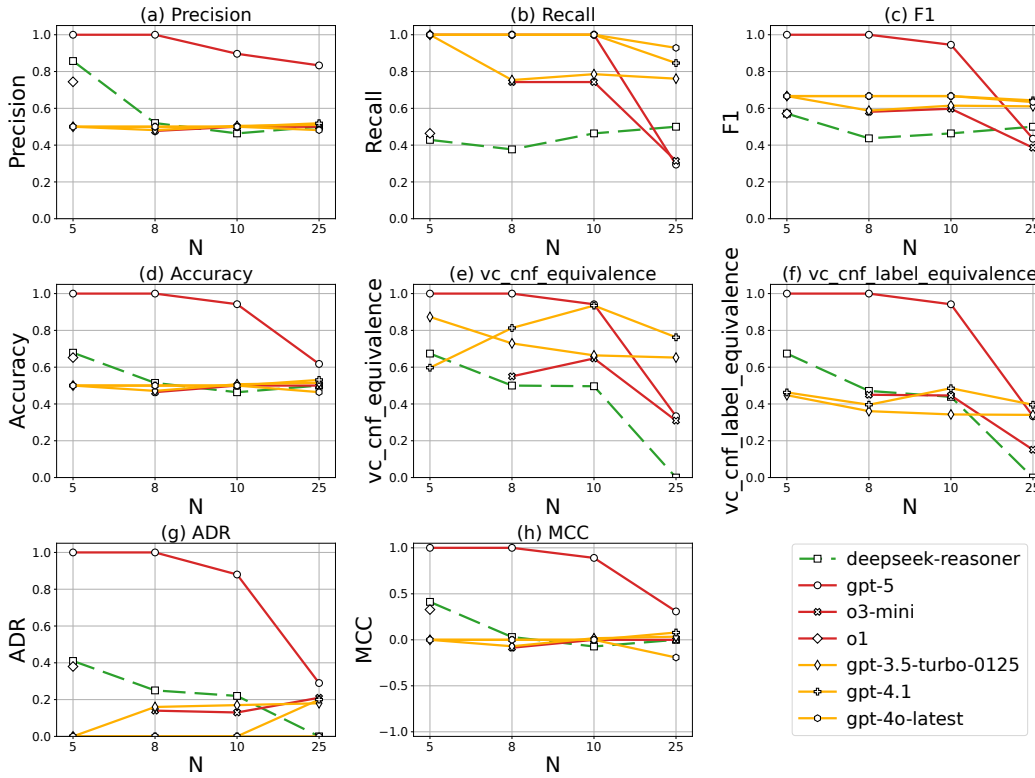


Fig. 1. Vertex Cover results across models. **(e)** *vc_cnf_equivalence*: CNF–graph decision agreement; typically > 80% for most models, indicating a stable decision rule across representations (independent of correctness). **(f)** *vc_cnf_label_equivalence*: triple agreement among CNF decision, graph decision, and ground truth; reflects true correctness.

- **`deepseek-reasoner`:** performs better than chance at small N, yet still trails `GPT-5` and loses reliability as instance size increases.

These outcomes show that simply re-expressing CNF as graphs does not make the decision task easier for current LLMs; genuine gains are model-dependent and fragile under scaling.

3.5 Cross-Representation Consistency

Prediction alignment across CNF and graphs (Figure 1 (e): vc_cnf_equivalence). Despite weak absolute accuracy for many models on graphs, their *decisions* are largely *consistent* across equivalent representations: for most models, the fraction of instances where the CNF prediction matches the Vertex-Cover prediction exceeds **80%**. This means that—even when they are not correct—models tend to preserve a stable decision rule when the SAT instance is recast as a graph, suggesting sensitivity to structure beyond the surface form of CNF.

Agreement with ground truth across both views (Figure 1 (f): vc_cnf_label_equivalence). Requiring *triple agreement* (CNF decision = graph decision = true label) reveals the boundary of this transfer. Most models remain near chance, indicating they are *consistently wrong across representations*. **Crucially, `GPT-5` is the exception:** its CNF and graph decisions are not only highly consistent

but also *correct* on small instances, yielding clearly non-random triple agreement. This provides direct evidence for representation-invariant *and* truth-aligned reasoning—exactly the behavior our research question seeks to detect.

Answer to the Research Question. LLMs *do* exhibit representation-invariant behavior: CNF and graph decisions largely agree (Figure 1(e)). For most models, the observed invariance primarily reflects adherence to a stable heuristic applied across representations. Notably, `GPT-5` exhibits representation invariance that is both consistent and accurate, answering our research question affirmatively: satisfiability judgments can remain stable across equivalent representations while aligning with the ground truth.

Finally, Figure 1(g) and (h) compare ADR and MCC under the paired evaluation scheme. As in prior sections, ADR isolates the true pair-level differentiation ability, while MCC is confounded by systematic biases. The degradation of both metrics with growing N reaffirms scalability as the dominant bottleneck across models.

3.6 Implications

- **SAT as a foundational problem.** The high CNF–graph decision alignment (Figure 1 (e)) shows that the logical structure captured by SAT transfers to Vertex Cover. This supports our thesis from Section 1 (Introduction): SAT encapsulates core invariants that underlie other reasoning tasks.
- **Invariance is useful only with base competence.** Figure 1 (f) makes clear that representation stability alone is insufficient: without strong base competence, models remain consistently wrong across representations. `GPT-5` stands out because it achieves both high invariance and correctness on small N, demonstrating genuine structure-sensitive reasoning rather than surface heuristics.
- **Scalability remains the bottleneck.** Even for `GPT-5`, performance degrades as N increases, echoing the trend on CNF inputs. To leverage representation-invariant reasoning reliably, models must improve scale robustness.

Most LLMs preserve their decision boundary when moving from CNF to graphs, but only `GPT-5` (on small instances) preserves a *correct* boundary. Thus, SAT indeed functions as a *foundational* problem: success on SAT not only predicts transfer to other reductions (e.g., Vertex Cover) but also reveals whether a model's invariance reflects genuine logical competence or merely stable yet incorrect heuristics.

4 3SAT Transfer to 3D Packing

4.1 Motivation.

As outlined in Section 1 (Introduction), 3SAT serves as a foundational probe of LLM reasoning. To test *problem-family transfer* beyond CNF and graphs, we map 3SAT instances into a discrete *3D packing* formulation, which represents another NP problem class, such that the feasibility of the packing instance directly corresponds to the satisfiability of the original formula. If models truly capture the underlying logical structure, their behavior on packing should echo their behavior on 3SAT, just as we observed for Vertex Cover.

4.2 Reduction and prompt.

Each 3SAT instance is reduced to a grid container with two object types and hard constraints: (i) per x-index, select exactly one of two fixed-orientation rods (at $y = 0$ or $y = 1$) spanning the full depth; (ii) for each depth layer z, place exactly one unit "token" at a coordinate from its `allowed_slots`; and (iii) a token is valid only if covered by the chosen rod at the same (x, y). Feasibility requires

that all tokens can be placed under these rules. We use a structured, one-line-JSON prompt that enforces format and validity checks:

```
def make_3d_packing_prompt(instance, instance_name=None):

    X, Y, Z = instance["container"]["size"]
    header = (
    "You are an expert in discrete 3D packing and constraint solving.\n"
    f"- Container (grid): X = {X}, Y = {Y}, Z = {Z}; rotations are NOT allowed and orientations are locked.\n"
    "- Objects come in TWO types (no other types exist):\n"
    "  * Type-A (long rods): for each x-index in {1..X} there are EXACTLY TWO rods, anchored at y=0 and y=1, "
    "each with size [1,1,Z] and spanning cells (x=const, y in {0,1}, z=0..Z-1). "
    "You must SELECT EXACTLY ONE rod per x-index (choose y=0 OR y=1), never both.\n"
    '  * Type-B (unit cubes, "tokens"): there is EXACTLY ONE Type-B object for each depth z=j (j=0..Z-1). '
    "Each Type-B object lists its allowed placements as coordinates in its `allowed_slots` field.\n"
    "- Valid placement rule:\n"
    "  * A Type-B placement at (x,y,z=j) is VALID ONLY if that cell is covered by a SELECTED Type-A rod at the same (x,y).\n"
    "  * You MUST choose coordinates ONLY from the `allowed_slots` provided; do not invent coordinates.\n"
    "- Feasibility means EVERY Type-B object can be placed validly under these rules.\n\n"
    "Return ONLY a ONE-LINE JSON object:\n"
    '  {"answer":"YES"|"NO",'
    ' "assignment":{"1":true/false,"2":true/false,...,"' + str(X) + '":true/false},'
    ' "tokens":[[x,y,z],...],'
    ' "explain":"<= 2 sentences"}\n'
    "Where:\n"
    '- `assignment` uses STRING KEYS for ALL x-indices 1..X; true => select the rod at y=0, false => select the rod at y=1.\n'
    "  - `tokens` must contain EXACTLY one coordinate for EACH depth layer z=0..Z-1, in ascending z order, "
    "and each coordinate must be one of that layer's `allowed_slots`.\n"
    "If ANY Type-B object cannot be validated, return NO with empty assignment and tokens.\n"
    "Do not include any extra text before or after the one-line JSON.\n"
    )

    return header
```

Listing 2. Prompt builder for 3D Packing

4.3 Example 3-SAT → 3D-packing (two clauses)

Consider the 3-CNF

$$\Phi = (x_1 \vee \neg x_2 \vee x_3) \wedge (\neg x_1 \vee x_2 \vee x_3).$$

There are $n = 3$ variables and $m = 2$ clauses. We construct the packing instance as follows.

Container (how the size is derived). We use a discrete grid of size $[X, Y, Z] = [n, 2, m] = [3, 2, 2]$:

- $X = n$ indexes variables along the x-axis ($x = 1, 2, 3$).
- $Y = 2$ encodes Boolean truth as layers: $y = 0$ means True, $y = 1$ means False.
- $Z = m$ indexes clauses along depth: $z = 0, 1$.

Type-A objects (rods): size and position, and where they come from. For each variable x_i we create two rods, one for True and one for False. Each rod spans *all* clause layers so that any clause token placed at depth z can be covered by the chosen truth-layer for that variable.

- **Size.** Every rod has size $[1, 1, m] = [1, 1, 2]$.
- **Anchors (positions).** Rods are axis-aligned and anchored at $z = 0$:

$$\text{True-rod of } x_i : \ (x = i,\ y = 0,\ z = 0), \qquad \text{False-rod of } x_i : \ (x = i,\ y = 1,\ z = 0).$$

- **Constraint.** *Exactly one* rod must be selected per variable (choose either the True-rod or the False-rod).

For this example ($i \in \{1, 2, 3\}$) the six rods are:

$$\text{rod\_T\_x1 at } (1, 0, 0),\ \text{rod\_F\_x1 at } (1, 1, 0),$$
$$\text{rod\_T\_x2 at } (2, 0, 0),\ \text{rod\_F\_x2 at } (2, 1, 0),$$
$$\text{rod\_T\_x3 at } (3, 0, 0),\ \text{rod\_F\_x3 at } (3, 1, 0),$$

each with size $[1, 1, 2]$ and extending over $z = 0, 1$.

Type-B objects (tokens): size, allowed slots, and how they are derived from clauses. We create one token per clause and restrict where it may be placed using the clause's literals.

- **Size.** Every token has size $[1, 1, 1]$ (occupies a single grid cell).
- **Allowed-slot rule.** For clause C_j ($j \in \{0, 1\}$):

$$x_i \implies (x = i,\ y = 0,\ z = j), \qquad \neg x_i \implies (x = i,\ y = 1,\ z = j).$$

- **Allowed slots for this example.**
  - $C_0 = (x_1 \vee \neg x_2 \vee x_3)$ at $z = 0$ gives
$$\{(1, 0, 0),\ (2, 1, 0),\ (3, 0, 0)\}.$$
  - $C_1 = (\neg x_1 \vee x_2 \vee x_3)$ at $z = 1$ gives
$$\{(1, 1, 1),\ (2, 0, 1),\ (3, 0, 1)\}.$$

Feasibility rule (semantic constraint). A packing is feasible iff *every* clause token is placed on one of its allowed slots *and* the chosen cell is covered by the rod selected for that variable at the same (x, y). Equivalently, feasibility of the packing is equivalent to satisfiability of Φ.

A SAT assignment and the corresponding feasible packing. The formula is satisfiable. For example, take

$$x_1 = \text{True}, \quad x_2 = \text{True}, \quad x_3 = \text{True}.$$

- **Selected rods (from the assignment).** Choose the True-layer rods
$$(x, y) = (1, 0),\ (2, 0),\ (3, 0),$$
  i.e., `rod_T_x1`, `rod_T_x2`, `rod_T_x3`, each of size $[1, 1, 2]$.
- **Token placements (one per clause depth).**
  - For C_0 at $z = 0$, pick $(1, 0, 0)$ (allowed by literal x_1 and covered by `rod_T_x1`).
  - For C_1 at $z = 1$, pick $(2, 0, 1)$ (allowed by literal x_2 and covered by `rod_T_x2`).

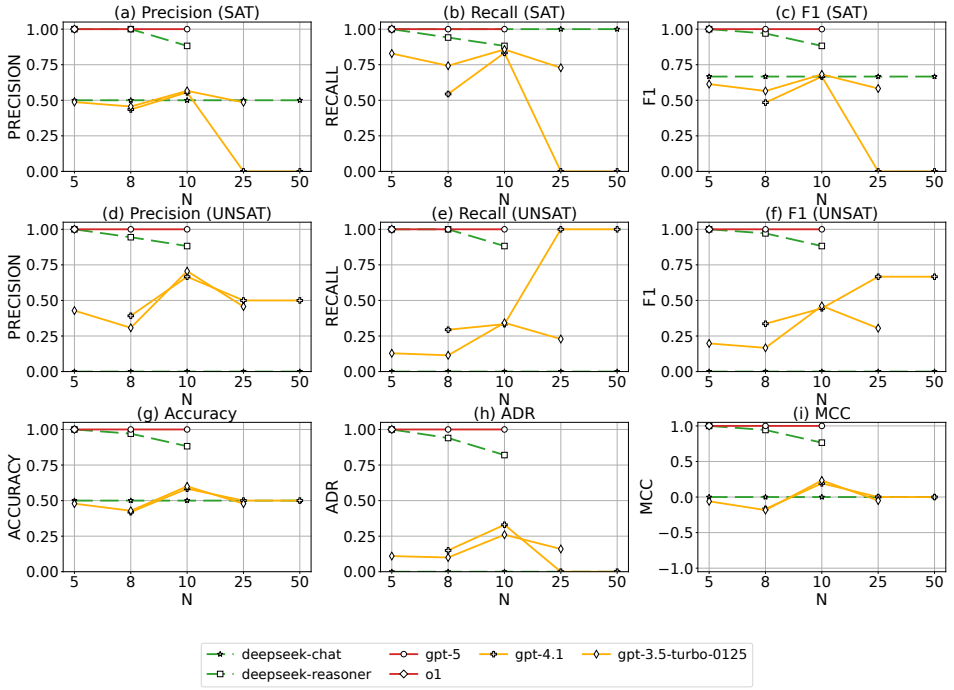


Fig. 2. 3D packing: predictions when LLM answers "yes" (3×3).

Both tokens use allowed slots and lie on selected rods, so the packing is feasible, matching the fact that Φ is SAT.

4.4 Experimental setup.

We convert the **70 paired** 3SAT instances (SAT/UNSAT) used in our prior experiment 3 in Section 5 (Refined Understanding with Paired Formulas) into 3D-packing instances and query the same set of LLMs with Listing 2. Models produce a categorical feasibility decision and, when answering `YES`, a concrete rod selection and token placements.

4.5 Analysis of Results

Figure 2 summarizes the 3D–packing results under the one-line–JSON protocol when the model must output a binary feasibility decision and, on "YES", a concrete assignment. The trends closely mirror those observed for CNF and Vertex Cover.

(i) Conventional metrics look strong at small N but collapse with scale. On the SAT side (panels (a)–(c)), *reasoning-oriented* models (e.g., `gpt-5`, `deepseek-reasoner`) attain near-perfect precision/recall/F1 for $N \leq 10$, then degrade sharply as N increases; non-reasoning baselines (`deepseek-chat`, etc.) hover around a mid-level plateau and never approach solver-like performance. On the UNSAT side (panels (d)–(f)), we see the usual asymmetry: some models gain recall at larger N by predicting NO more often, but this comes with low precision, yielding unstable F1. These patterns reproduce the class-skew sensitivities discussed in Section 3 (INITIAL MISAPPREHENSION, UNDER TRADITIONAL METRICS).

(ii) Accuracy and MCC are poor diagnostics; ADR is the most informative. Overall accuracy (panel (g)) remains near chance for several models even when one class is systematically mishandled, confirming that accuracy alone hides failure modes. MCC (panel (i)) is positive for small N but quickly drifts toward 0 as N grows; as analyzed in Section 2 of this supplemental material, MCC mixes joint correctness with class imbalance and can look acceptable while pairwise discrimination is failing. By contrast, *ADR* (panel (h)) gives a clean signal: reasoning models exhibit high ADR at small N (i.e., they simultaneously get both members of a SAT/UNSAT pair right), followed by a monotone decline toward chance with increasing N. This agrees with the mathematical properties in Section 1 and with the CNF/VC findings in Section 3 of this supplemental material.

(iii) Problem-family transfer and representation invariance. The scaling profile in packing echoes 3SAT and Vertex Cover: strong small-N behavior for reasoning models, rapid convergence to chance at larger N, and persistent SAT bias for non-reasoning models. This confirms *problem-family transfer*: models port both their strengths and their weaknesses from CNF to packing. Moreover, high ADR coincides with higher *Packing-Assignment Validity* (when "YES" is returned), indicating that pairwise discrimination aligns with producing logically valid witnesses—a representation-robust sign of structural reasoning.

4.6 Implications

The packing formulation preserves the essential difficulty landscape of 3SAT. Traditional metrics can overstate competence due to class imbalance and prediction bias, whereas ADR (together with witness checking) faithfully tracks reasoning capacity and its scale-induced degradation. These results reinforce our thesis that **SAT is a foundational probe**: success (or failure) on SAT predicts transfer to other NP-style reductions such as 3D packing.

5 Cross-task agreement statistics and ADR relations

To compare model behavior between SAT (or 3SAT) and an equivalent reduced task (e.g., Vertex Cover or 3D packing), we report *instance-level cross-task agreement* statistics in addition to task-specific metrics (Accuracy/F1/MCC) and paired metrics (ADR).

5.1 Instance correspondence and label mapping.

Let x denote a source SAT-family instance (SAT or 3SAT), and let $R(x)$ denote its deterministically constructed counterpart in the reduced task (e.g., Vertex Cover or 3D packing). We compare predictions after applying the canonical label mapping

$$\mathrm{SAT} \leftrightarrow \mathtt{YES}, \qquad \mathrm{UNSAT} \leftrightarrow \mathtt{NO}.$$

For convenience, write

$$\hat{y}_{\mathrm{SAT}}(x) \in \{\mathrm{SAT}, \mathrm{UNSAT}\}, \qquad \hat{y}_{\mathrm{task}}(R(x)) \in \{\mathtt{YES}, \mathtt{NO}\},$$

and let $\tilde{y}_{\mathrm{task}}(R(x)) \in \{\mathrm{SAT}, \mathrm{UNSAT}\}$ denote the mapped task-side prediction.

Let $y(x) \in \{\mathrm{SAT}, \mathrm{UNSAT}\}$ be the ground-truth label of x; by construction of the reduction, the reduced instance $R(x)$ has the same truth value under the mapped label space.

5.2 Instance-level same-prediction rate.

For a fixed model m, instance size bucket N, and a set of aligned instances $\mathcal{D}_{m,N}$, we define the cross-task same-prediction indicator

$$I_{\mathrm{same}}(x) = \mathbf{1}\{\hat{y}_{\mathrm{SAT}}(x) = \tilde{y}_{\mathrm{task}}(R(x))\}\,.$$

The *instance-level same-prediction rate* is

$$\mathrm{SamePred}(m,N) = \frac{1}{|\mathcal{D}_{m,N}|} \sum_{x \in \mathcal{D}_{m,N}} I_{\mathrm{same}}(x).$$

This is the statistic summarized in the main text as, e.g., "SAT/VC same prediction" or "3SAT/packing same prediction" (after aggregation; see below).

5.3 Conditional correctness on disagreement.

We next examine which side is more often correct when the two predictions disagree. Define the disagreement set

$$\mathcal{D}^{\neq}_{m,N} = \left\{ x \in \mathcal{D}_{m,N} : \hat{y}_{\mathrm{SAT}}(x) \neq \tilde{y}_{\mathrm{task}}(R(x)) \right\}.$$

On this set, the SAT-side correctness rate is

$$\mathrm{CorrSAT} \mid\neq (m,N) = \frac{1}{|\mathcal{D}^{\neq}_{m,N}|} \sum_{x \in \mathcal{D}^{\neq}_{m,N}} \mathbf{1}\{\hat{y}_{\mathrm{SAT}}(x) = y(x)\},$$

and the reduced-task-side correctness rate is

$$\mathrm{CorrTask} \mid\neq (m,N) = \frac{1}{|\mathcal{D}^{\neq}_{m,N}|} \sum_{x \in \mathcal{D}^{\neq}_{m,N}} \mathbf{1}\{\tilde{y}_{\mathrm{task}}(R(x)) = y(x)\}.$$

Because the two predictions disagree, exactly one side can be correct (under deterministic labels), so

$$\mathrm{CorrSAT} \mid\neq (m,N) + \mathrm{CorrTask} \mid\neq (m,N) = 1$$

up to rounding error in reported percentages.

5.4 Aggregation rule ("all LLMs accumulated").

Unless otherwise stated, the percentages reported in the main text for cross-task agreement (e.g., 75.3%, 73%/27%, 80.2%, 53%/47%) are computed by *micro-aggregation*: we pool all eligible aligned instances across all evaluated models and all reported N buckets, and then compute the above rates on the pooled set. Equivalently, if $\mathcal{U} = \bigcup_{m,N} \mathcal{D}_{m,N}$ denotes the pooled multiset of (model, N, instance) evaluation records, then

$$\mathrm{SamePred}^{\mathrm{micro}} = \frac{\sum_{(m,N)} \sum_{x \in \mathcal{D}_{m,N}} I_{\mathrm{same}}(x)}{\sum_{(m,N)} |\mathcal{D}_{m,N}|},$$

and similarly for the conditional correctness rates on the pooled disagreement set. We use micro-aggregation to reflect the aggregate empirical frequency across all evaluated runs. (If a macro-average is reported in future versions, we will state it explicitly.)

5.5 Relation to ADR comparisons across tasks.

The cross-task agreement statistics above are *instance-level* and compare whether two task representations induce the same prediction on corresponding instances. By contrast, ADR is a *pair-level* metric computed separately within each task:

$\mathrm{ADR}_{\mathrm{SAT}}$ on paired SAT/UNSAT instances,

$\mathrm{ADR}_{\mathrm{task}}$ on the corresponding paired reduced-task instances.

In our experiments, the reduced-task paired dataset is generated by applying the same deterministic reduction instance-wise to the source paired SAT-family dataset, so pair correspondence is preserved across tasks (i.e., the task-side pair is the image of the source-side pair under R). Therefore, ADR

comparisons such as "GPT-5 has ADR 0.79 on SAT but ∼0.27 on Vertex Cover at $N = 25$" compare matched pair families under different representations, rather than unrelated datasets.

5.6 Empirical summary on the evaluated models.

Using the micro-aggregation rule in Section 5, we summarize the cross-task agreement statistics across all evaluated models and reported N buckets as follows.

Vertex Cover (CNF ↔ VC).. Across all matched instances, cross-task agreement is more common than disagreement: 2131/2830 instances agree (**75.30%**), while 699/2830 disagree (**24.70%**). Among the agreement cases, the shared prediction is correct in **65.60%** of instances and incorrect in **34.40%**, indicating that agreement alone does not imply correctness. Among the disagreement cases, the CNF-side prediction matches the ground-truth label in **72.96%** of instances, whereas the VC-side prediction matches the label in only **27.04%**. Thus, when CNF and VC predictions diverge, the CNF side is substantially more likely to be correct.

3D Packing (3SAT ↔ packing). Across all matched instances, agreement is again more common than disagreement: 1290/1609 instances agree (**80.17%**), while 319/1609 disagree (**19.83%**). Among the agreement cases, the shared prediction is correct in **60.23%** of instances and incorrect in **39.77%**. Among the disagreement cases, the 3SAT-side prediction matches the ground-truth label in **52.98%** of instances, while the packing-side prediction matches the label in **47.02%**. Unlike Vertex Cover, the disagreement pattern here is nearly balanced, indicating that neither side dominates strongly when the two representations induce different predictions.

Cross-task comparison. Across both transformations, predictions agree on most matched instances (**75.3%** for Vertex Cover and **80.2%** for 3D packing), showing substantial cross-representation stability. However, the disagreement analysis reveals different error structures: Vertex Cover disagreements strongly favor the CNF side (about **73%** vs. **27%**), whereas 3D-packing disagreements are close to balanced (about **53%** vs. **47%**). This difference is consistent with the main-text interpretation that high cross-task agreement can reflect either genuine transfer or stable but task-sensitive failure modes, which is why ADR and task-specific correctness metrics remain necessary.

5.7 Reporting note.

These agreement statistics should be interpreted jointly with ADR: high same-prediction rates can arise from *correct cross-representation transfer* or from *consistently wrong but stable heuristics*. ADR and task-specific correctness metrics are therefore necessary to disambiguate representation invariance from genuine reasoning competence.

6 Packing and Vertex Cover Prompts

6.1 Full Vertex Cover Prompt Template

Listing 3 shows the complete version of the Vertex Cover prompt. Unlike the abbreviated form in the main text (Listing 1), here we retain all formatting guards, edge-case instructions, and multiple valid output examples.

```
def make_vc_prompt(V, E, k, instance_name=None):
    """
    V: list (sorted)
    E: list of 2-tuples (sorted, endpoints sorted)
    k: int
    return: prompt string
    """
```

```
    V_str = "{ " + ", ".join(str(v) for v in V) + " }"
    E_str = "{ " + ", ".join("{" + f"{u}, {v}" + "}" for (u, v) in E) + " }"

    header = (
        "You are an expert on the graph vertex cover problem. "
        "The following is an undirected graph represented as a pair where "
        "the first component is the set of vertices and the second is a set of edges. "
        f"Given this graph, determine whether there exists a vertex cover of size <= {k}.\n\n"
        "IMPORTANT OUTPUT FORMAT:\n"
        "Return ONLY a single-line JSON object with keys:\n"
        '  - "answer": either "YES" or "NO"\n'
        '  - "cover": a list of distinct integers (the vertex set) if and only if "answer" is "YES"; otherwise []\n'
        '  - "explain": a short plain-text explanation (max 1-2 sentences)\n'
        "Rules:\n"
        f"  * Output \"YES\" ONLY IF you can provide a concrete vertex set C with |C| <= {k} that covers EVERY edge in E.\n"
        "  * If you are unsure OR cannot verify ALL edges are covered OR cannot list such C, output \"NO\" and set cover=[].\n"
        "  * No text outside the JSON. One line only.\n"
        "Example valid outputs:\n"
        '{"answer":"YES","cover":[1,3,7],"explain":"All edges incident to at least one of 1,3,7."}\n'
        f'{{"answer":"NO","cover":[],"explain":"Unable to verify a cover within size {k}."}}\n'
    )

    name_line = (f"Instance: {instance_name}\n" if instance_name else "")
    body = (
        f"{name_line}"
        "Graph (V, E):\n"
        f"V = {V_str}\n"
        f"E = {E_str}\n"
        f"k = {k}\n\n"
        "Task: Decide if there exists a vertex cover of size <= k.\n"
        "If and only if YES, return a valid cover set and a brief explanation in the JSON as described."
    )
    return header + "\n" + body
```

Listing 3. Full prompt builder for Vertex Cover instances (full version).

7 Pair Generation via MaxSAT Repair (RC2)

This supplemental material documents an auxiliary pair-generation procedure used to construct a satisfiable counterpart for an **UNSAT** CNF instance. Given an **UNSAT** formula F in CNF, we compute a minimally-deleted satisfiable formula F^{SAT} by solving a *MaxSAT repair* problem [Biere et al. 2009; Gu et al. 1997]: we introduce one relaxation variable per clause and allow the solver to "disable" clauses at unit cost. We then remove exactly those clauses whose relaxation variables

are set to true in the MaxSAT model. The resulting repaired CNF is verified satisfiable by an independent SAT solver (MiniSat 2.2) [Eén and Sörensson 2003].

7.1 Disambiguation: how this relates to the paired instances in the main paper.

In the *main evaluation* (Section 5), paired instances are defined under a *single-edit* model (literal flip / literal replacement / single-clause deletion) and are *solver-verified* to flip satisfiability. The RC2-based repair described here is *not* a second definition of pairing; rather, it is an *implementation mechanism* that produces a satisfiable counterpart under a different edit model—*clause deletions*—and provides a quantitative notion of "repair distance" via the RC2 cost. When we require strict single-edit pairs, we either (i) restrict to cases where the optimal RC2 cost is 1 (single-clause deletion), or (ii) treat higher-cost repairs as feasibility-flipping pairs under the clause-deletion edit model and report the RC2 cost alongside ADR to avoid mixing edit models in interpretation.

7.2 MaxSAT formulation.

Let $F = \bigwedge_{i=1}^{m} C_i$ be an **UNSAT** CNF with n original variables. For each clause C_i, we add a fresh relaxation variable r_i and create a soft clause $(C_i \vee r_i)$ with weight 1. Intuitively, setting $r_i = \text{true}$ allows the solver to satisfy the clause regardless of C_i's literals, effectively "skipping" C_i. The RC2 solver returns a model minimizing the total weight (here, the number) of activated relaxation variables [Berg et al. 2024; Ignatiev et al. 2018]. We then construct F^{SAT} by deleting all clauses C_i whose r_i is activated.

7.3 Relation to "minimal edit" pairing.

This repair procedure yields a **SAT** counterpart at minimal *clause-deletion cost* among all repairs under our objective (minimize the number of deleted clauses) [Biere et al. 2009; Gu et al. 1997]. In the main paper, "minimal edit" refers to edit distance one under the single-edit model (literal flip / literal replacement / clause deletion). RC2 instead provides minimality under the *clause-deletion* edit model. We keep these notions separate: ADR is always computed on the paired dataset defined by the relevant edit model, and we report RC2 costs to characterize how far **UNSAT** instances are from satisfiable under clause deletions.

7.4 Implementation notes (correspondence to code).

Our implementation uses `PySAT` [Ignatiev et al. 2018]: (i) parse the DIMACS CNF file into a clause list, (ii) construct `WCNF` by appending $(C_i \vee r_i)$ with unit weight, (iii) generate r_i using `IDPool(start_from = nv+1)` to avoid conflicts with original variables, (iv) solve via `RC2.compute()` and read the optimal model, (v) delete exactly those clauses whose corresponding relaxation variables appear in the model, and (vi) re-check satisfiability of the repaired CNF with `Minisat22` (MiniSat 2.2) [Eén and Sörensson 2003].

7.5 Statistics to report.

For reproducibility and to characterize repair distance under clause deletions, we report (or recommend reporting) the following per setting: (i) the distribution of RC2 costs (number of deleted clauses), (ii) the fraction of instances with optimal cost 1 (single-clause deletion), (iii) average/-median cost by (N, α) bucket, and (iv) the empirical relationship between repair cost and ADR degradation (reported without mixing edit models).

8 SAT-derived benchmark: SATLIB "Flat" graph-coloring

To complement the main CNF-based experiments, we conducted a small supplementary validation on a SAT-derived benchmark built from SATLIB "Flat" graph-colouring instances [Culberson and

Algorithm 1: Pair generation for **UNSAT** CNFs via MaxSAT repair using RC2.

Input : UNSAT CNF F with clauses $\{C_1, \ldots, C_m\}$ and n variables
Output : A satisfiable CNF F^{SAT} obtained by deleting a minimum number of clauses, and the deleted clause index set $\mathcal{D}$

Build weighted CNF.
Create an empty weighted CNF W.
Create an ID pool starting from $n+1$ to generate fresh relaxation variables.
for $i \leftarrow 1$ **to** m **do**
  $r_i \leftarrow$ NewVar() `// fresh relaxation variable not in F`
  Add soft clause $(C_i \vee r_i)$ to W with weight 1.

Solve MaxSAT.
Run RC2 on W to obtain an optimal model M minimizing the number of activated r_i [Berg et al. 2024; Ignatiev et al. 2018].
Let $\mathcal{D} \leftarrow \{\, i \mid r_i \in M \,\}$ `// clauses to delete`

Construct repaired CNF.
Initialize $F^{\mathrm{SAT}} \leftarrow \bigwedge_{i \notin \mathcal{D}} C_i$.

Independent verification.
Verify SAT(F^{SAT}) = true using MiniSat 2.2 [Eén and Sörensson 2003]; otherwise flag the instance as failed.

return $(F^{SAT}, \mathcal{D})$

Gent 2001; Hoos and Stützle 2000]. The goal of this experiment is *not* to establish a new benchmark result, but to test whether the paired-evaluation logic underlying ADR remains informative on a non-CNF-native problem representation (graph colouring) that can be translated to SAT.

8.1 Purpose and scope.

This experiment serves as a proof-of-concept for the transferability of ADR beyond randomly generated CNF formulas. In particular, we test whether the discrepancy between conventional single-instance metrics and ADR persists when paired instances are constructed at the *graph level* (via small edge edits) and then encoded as SAT for solving.

8.2 Source benchmark.

We used the SATLIB "Flat" graph-colouring benchmark, specifically the `flat30-60` set [Culberson and Gent 2001; Hoos and Stützle 2000]. In the SATLIB benchmark listing, `flat30-60` consists of graphs with 30 vertices and 60 edges, with 100 instances in the original release, and is listed as *all satisfiable*. We use these instances as *base graphs* only.

This detail is important for interpretation: the **UNSAT** instances used in our ADR evaluation are *not* taken directly from SATLIB. Instead, they are generated by our graph-level pairing procedure (described below) via small edge edits and then certified by deterministic checking/solving.

8.3 Paired-instance construction (graph level).

Starting from a base graph instance G, we construct a paired instance G' by a small edge edit (addition or deletion), with the aim of flipping k-colourability while keeping the perturbation

Table 1. Supplementary result on a SAT-derived paired benchmark constructed from SATLIB `flat30-60` (30 vertices, 60 edges; 100 original base instances, all satisfiable in the SATLIB release). Paired SAT/**UNSAT** instances are generated by our graph-level edge-edit procedure and certified before evaluation.

Model	Dataset	N	Precision	Recall	F1	Accuracy	MCC	ADR
`GPT-5`	SATLIB Flat (`flat30-60`)	30	0.7766	0.7604	0.7684	0.7708	0.5418	0.6000

minimal in the graph domain. Concretely, we apply a bounded search over candidate edge edits and retain only those pairs (G, G') that satisfy all of the following conditions:

(1) G and G' differ by a small graph edit (in our implementation, a single edge addition/removal in the accepted pair set);
(2) exactly one of G and G' is k-colourable (feasibility-flipping pair);
(3) the feasibility labels are certified by a deterministic checker/solver.

This yields graph-level counterparts that are minimally perturbed under our edit model while enabling deterministic verification of the feasibility flip. In particular, because the `flat30-60` base set is all satisfiable, the paired construction is the mechanism that introduces certified **UNSAT** counterparts for ADR evaluation.

8.4 SAT encoding and verification.

Each graph-colouring instance (both G and G') is translated into a SAT formula using a standard k-colouring-to-SAT encoding (one-hot vertex-colour variables with clause constraints for colour assignment and edge conflicts), i.e., a classical polynomial-time reduction from graph colouring to SAT [Karp 2009]. We then use a symbolic SAT solver to verify the satisfiability status of the encoded formulas and to certify the feasibility-flipping property of each pair before querying the LLM.

8.5 LLM evaluation protocol.

We evaluated `GPT-5` on the resulting SAT instances using the same prompting and output-parsing protocol used in the main paired CNF experiments (i.e., the same decision-only evaluation logic, external verification of satisfiability labels, and the same paired-analysis pipeline where applicable). This keeps the supplementary experiment methodologically aligned with the main study, so that any observed gap between conventional metrics and ADR is attributable to the paired-evaluation setting rather than a different prompting regime.

8.6 Metrics.

For comparability with the main experiments, we report both:

- **Conventional single-instance metrics**: Precision, Recall, F1, Accuracy, and MCC, computed over individual instances in the supplementary dataset; and
- **ADR**: computed over feasibility-flipping pairs, where a pair is counted as correct only if the model correctly classifies *both* members of the pair.

As in the main paper, ADR captures joint discrimination performance under minimal perturbations and is therefore stricter than per-instance aggregate metrics.

8.7 Interpretation.

This supplementary result provides additional evidence that the value of ADR is not tied to raw CNF syntax. Even when instances originate from a graph problem and are paired via graph-level edits before SAT translation, the same pattern emerges: single-instance metrics may overstate

capability, whereas ADR more directly reflects whether the model reliably distinguishes minimally perturbed counterparts.

A further point is that this validation starts from a benchmark subset whose original instances are all satisfiable (`flat30-60` in SATLIB). The observed ADR gap therefore does not arise from a pre-balanced benchmark design, but persists even when SAT/UNSAT pairing is introduced through a controlled, verified perturbation procedure at the graph level.

8.8 Limitations.

This is a *small-scale supplementary validation*, not a full benchmark study. It does not aim to characterize performance across all SATLIB graph-colouring families, parameter settings, or graph sizes, nor does it optimize pair-generation procedures for maximal coverage. The result should therefore be read as evidence of *portability of the evaluation framework* (ADR + paired analysis), rather than as a definitive performance study on graph colouring.

8.9 Reproducibility notes (recommended items to report).

To facilitate replication, the following implementation details should be reported alongside the artifact or supplementary materials:

- the exact SATLIB subset used (here: `flat30-60`), including the SATLIB-listed base properties (30 vertices, 60 edges, 100 instances, all satisfiable);
- the target number of colours k used in the colouring feasibility test;
- the edge-edit search policy (addition/deletion candidates, acceptance criteria, and maximum attempts per base graph);
- the number of accepted pairs and the corresponding number of individual instances evaluated;
- the SAT encoding variant for graph colouring;
- the deterministic solver/checker used for label certification;
- the prompt template / decoding settings (if they differ from the main experiments).

References

Jeremias Berg, Matti Järvisalo, Ruben Martins, Andreas Niskanen, and Tobias Paxian. 2024. Maxsat evaluation 2024: Solver and benchmark descriptions. (2024).

Armin Biere, Marijn Heule, and Hans van Maaren. 2009. *Handbook of satisfiability*. Vol. 185. IOS press.

Davide Chicco and Giuseppe Jurman. 2020. The advantages of the Matthews correlation coefficient (MCC) over F1 score and accuracy in binary classification evaluation. *BMC genomics* 21, 1 (2020), 6.

Joseph Culberson and Ian Gent. 2001. Frozen development in graph coloring. *Theoretical computer science* 265, 1-2 (2001), 227–264.

Niklas Eén and Niklas Sörensson. 2003. An extensible SAT-solver. In *International conference on theory and applications of satisfiability testing*. Springer, 502–518.

Jun Gu, Paul W Purdom, John Franco, and Benjamin W Wah. 1997. Algorithms for the satisfiability (SAT) problem: A survey. *DIMACS series in discrete mathematics and theoretical computer science* 35 (1997), 19–151.

Holger H Hoos and Thomas Stützle. 2000. SATLIB: An online resource for research on SAT. *Sat* 2000 (2000), 283–292.

Alexey Ignatiev, Antonio Morgado, and Joao Marques-Silva. 2018. PySAT: A Python toolkit for prototyping with SAT oracles. In *International Conference on Theory and Applications of Satisfiability Testing*. Springer, 428–437.

Richard M Karp. 2009. Reducibility among combinatorial problems. In *50 Years of Integer Programming 1958-2008: from the Early Years to the State-of-the-Art*. Springer, 219–241.

Christopher D Manning. 2008. *Introduction to information retrieval*. Syngress Publishing,.

Brian W Matthews. 1975. Comparison of the predicted and observed secondary structure of T4 phage lysozyme. *Biochimica et Biophysica Acta (BBA)-Protein Structure* 405, 2 (1975), 442–451.

David MW Powers. 2020. Evaluation: from precision, recall and F-measure to ROC, informedness, markedness and correlation. *arXiv preprint arXiv:2010.16061* (2020).

Received 2025-09-04; accepted 2026-03-24